

VV WORK

Тестове завдання — Frontend Developer

Про проєкт

VV Work — платформа для пошуку роботи та працівників у Європі.

Ми створюємо сервіс, який має спростити процес пошуку роботи для кандидатів та пошуку працівників для роботодавців. Зараз ми формуємо команду й працюємо над MVP платформи.

Готових Figma-макетів для цієї частини платформи ще немає — дизайн буде розроблятися паралельно. Тому в цьому тестовому кандидат сам відповідає і за базові UI/UX-рішення, і за їх реалізацію в коді. Це навмисно: нам потрібна людина, яка не просто "верстає по пікселях", а вміє самостійно приймати й аргументувати інтерфейсні рішення.

Ваше завдання

Спроектувати (у коді, без Figma) і зверстати три сторінки платформи VV Work за описаним нижче брифом:

- Головна (/)
- Сторінка партнера / роботодавця (/partners/:slug) — динамічна
- Контакти (/контакти)

Header і Footer — спільні компоненти, перевикористовуються на всіх сторінках.

Бриф описує зміст і логіку блоків, не візуальний дизайн. Стиль, компоновання, кольори (за винятком базового бренд-акценту, який можна обрати самостійно) — на розсуд кандидата.

Header і навігація

Передбачте навігацію: логотип VV Work, Знайти роботу, Знайти працівника, Про нас, Партнери, Контакти — це приклад, структуру можна змінювати.

Hero та пошук вакансій

Головний екран має одразу пояснювати цінність платформи (наприклад: «Знайди роботу. Знайди працівника. Працюй у Європі.» — але текст можна придумати власний).

На сторінці партнера реалізуйте список вакансій із:

- Пошуком за назвою (ручний debounce, без бібліотек)
- Фільтром за категорією
- Коректною роботою обох фільтрів одночасно, без зайвих ре-рендерів списку

Дані та мережа

Дані вакансій і партнерів підвантажуються через fetch-обгортку, що симулює реальний API:

- Затримка відповіді 300–800мс (рандомна)
- Випадкова помилка запиту — приблизно 1 з 5
- Skeleton-стан під час завантаження, retry-блок при помилці — на кожній сторінці з асинхронними даними



Форма заявки

Валідація на клієнті: ім'я (мін. 2 символи), телефон/telegram (формат), повідомлення (опційно, до 500 символів).

- Помилки — інлайн, без alert()
- Сабміт — оптимістичний UI, запит теж іде через мокову fetch-обгортку з рандомною помилкою



Категорії професій

Блок популярних категорій (Будівництво, Виробництво, Логістика, Готельно-ресторанна сфера, IT, Водії, Інші — приклад, можна змінювати). Натискання на категорію фільтрує список вакансій на сторінці партнера.



Блок для роботодавців

Окремий блок на Головній: «Потрібні працівники?» з коротким поясненням переваг VV Work і СТА «Знайти працівника». Власне UX-рішення тут вітається.



Mobile-версія

Адаптивність 1440 → 768 → 375px. Брейкпоінти, макет і перебудову блоків під мобільний екран кандидат визначає повністю самостійно — це і є частина оцінюваного UX-рішення.



Footer

Базовий footer: VV Work, контакти, про компанію, для кандидатів, для роботодавців, соцмережі, політика конфіденційності, умови використання — структуру можна змінювати.



Важливо — проявіть власне бачення

Це не завдання зі сліпого виконання ТЗ. Немає єдиного правильного рішення — ви можете змінювати порядок блоків, структуру навігації, назви кнопок, підхід до карток вакансій, запропонувати власний UX-сценарій.

Можете:

- змінити структуру компонентів, якщо ваш варіант зручніший для масштабування
- запропонувати інший підхід до пошуку/фільтрації вакансій
- додати корисну функціональність, якої немає в брифі

- аргументовано відхилитися від деталей ТЗ там, де бачите краще рішення

Нам важливо побачити: як ви думаєте → як знаходите проблему → як приймаєте UI/UX і технічне рішення.



Мої рішення

Додайте до README короткий блок «Мої рішення»: 3–5 пунктів. Наприклад:

- Чому обрали таку структуру Головної сторінки?
- Як допомагаєте користувачу швидше знайти вакансію?
- Чому обрали такий підхід до стейт-менеджменту без Redux/Zustand?
- Як саме вирішили проблему зайвих ре-рендерів у списку вакансій?
- Що змінили в наданому брифі і чому?



Вимоги до роботи

Стек: Vite + React + TypeScript + Tailwind CSS (без сторонніх UI-кітів, без Redux/Zustand).

Бажано:

- TypeScript strict mode, без any
- Компонентна архітектура з чітким поділом відповідальності
- React Router, спільні Header/Footer
- Unit-тести (Jest/Vitest + Testing Library) на debounce-пошук, валідацію форми, retry-логіку — мінімум 60% покриття логіки
- Lighthouse Performance ≥ 90 на Головній у продакшн-білді
- Проходження базової accessibility-перевірки (axe) без critical/serious помилок

Код має бути зрозумілим іншому розробнику, який продовжить роботу над цим репозиторієм.



Час виконання

Термін виконання: 4 дні з моменту отримання завдання.

Не потрібно витрачати весь вільний час на тестове — ми оцінюємо підхід і якість інженерних рішень, а не кількість годин.



Що потрібно надіслати

- Посилання на публічний GitHub-репозиторій з осмисленою git-історією
- Посилання на деплой (Vercel) — обов'язково
- README: інструкція запуску, опис архітектури, скріншот Lighthouse, короткий опис unit-тестів
- Блок «Мої рішення» (3–5 пунктів)
- Якщо відхилялись від наданого брифу — коротко поясніть, що саме і чому



Як ми будемо оцінювати

Критерій	Що оцінюємо
UX	Логіка й зручність користування, самостійно продумана структура
UI	Візуальна якість, попри відсутність готового макета
Адаптивність	Якість перебудови під tablet і mobile
Архітектура	Компонентна структура, перевикористання Header/Footer, спільна логіка сторінок
Асинхронність	Обробка завантаження, помилок мережі, retry-логіка
Продуктивність	Lighthouse-показник, відсутність зайвих re-рендерів
TypeScript	Строга типізація, відсутність any, чистота коду
Тестування	Наявність і якість unit-тестів на ключову логіку
Доступність	Відповідність базовим вимогам accessibility
Git / процес	Історія комітів, README, деплой
Ініціативність	Власні UI/UX та технічні рішення, аргументовані відхилення від брифу



Головне

Не намагайтесь просто "зробити щоб працювало".

Подумайте про реального користувача і про розробника, який продовжить цей код після вас.

Ми оцінюємо не тільки результат. Ми оцінюємо те, як ви думаєте й будете рішення.

Успіхів! 